

When Self-Reported Fitness Becomes the Target: Independent Audits in LLM-Driven Recipe Evolution

Qindong Gan
Waseda University
ganqindong@fuji.waseda.jp

September 2026

Abstract

LLM-driven program evolution requires a score for deciding which generated programs reproduce. That score need not be comparable when each program controls part of its own measurement. We study this failure mode in a small reinforcement-learning system where an LLM rewrites training recipes for a fixed LunarLander policy trainer. Every trained policy receives both a training score and a harness-owned audit: 32 fresh-seed episodes scored only by raw environment reward, with no recipe-code execution. We compare audit-based against training-score-based selection in two reward-source protocols, using three independent 15-candidate outer loops per cell. With candidate-shaped training reward, claimed scores saturated at 10^9 and the final audited return averaged 4.0 under audit selection versus -762.1 under claimed selection; claimed-selection regret averaged 710.4. A fixed-raw control removed candidate reward shaping and score saturation. The gap narrowed but did not disappear: final audited return averaged -19.4 versus -137.1 , and claimed-selection regret averaged 110.7, with a selection error in all three runs. The comparison identifies candidate-defined reward scale as a major amplifier, not the only source, of self-report misranking. This is a reproducible mechanism-and-ablation case study, not a population estimate: each cell has only three runs, all in one environment with one model configuration, and the two protocols were run at different times.

1 Introduction

Large language models increasingly propose programs inside search loops: mathematical programs, algorithms, prompts, reward functions, and components of automated research systems [6, 7, 9, 11, 12, 17]. These systems depend on a selection score. With an exact verifier or a fixed benchmark harness, candidates share a trusted objective. With training programs, however, a tempting shortcut is to archive the best score reported during each candidate’s own run.

Self-reporting is dangerous when candidates can change reward shaping, evaluation horizon, sample count, or seeds. The score then mixes policy quality with a candidate-specific unit of measurement. Selection pressure can favor the report rather than the underlying policy—a direct Goodhart-style failure [8] related to specification gaming and reward hacking [1, 3, 5, 10, 15]. Unlike an agent that edits evaluator code, the candidates in our system obey their declared interface. The problem is more basic: the interface permits each candidate to define a different proxy scale.

We test the smallest structural remedy: keep candidate-controlled training, but add a separate evaluation path that receives only the trained policy parameters, uses fresh seeds, and scores raw environment reward. We call this path an *audit*. The experiment changes only which score drives parent selection and LLM feedback. Audits are still recorded in both selection conditions. We then

repeat the comparison with candidate reward shaping disabled, so that the training score is raw environment reward while candidates can still choose optimizer and evaluation schedules.

Our contributions are:

1. a compact, open implementation of audit-gated LLM recipe evolution with immutable run configuration, prompt hashing, resumable outer-loop randomness, and per-episode audit records;
2. twelve LunarLander outer-loop runs in a 2×2 design—audit versus training-score selection, crossed with candidate-shaped versus fixed-raw training reward—with three independent runs and 15 candidates per cell;
3. a tie-aware analysis showing that claim-based selection becomes scale-saturated under candidate shaping, while avoiding the unsupported assertion that every highly shaped candidate is a poor policy; and
4. a fixed-raw ablation showing substantially smaller but persistent selection errors after candidate reward content and scale are removed, plus reproducibility bundles containing all 180 candidate recipes, trained parameter vectors, audit records, and provenance metadata.

The study is intentionally narrow. Three runs per cell support a reproducible case study and effect-size description, not formal inference or broad claims about LLMs, reinforcement learning, or program evolution.

2 System

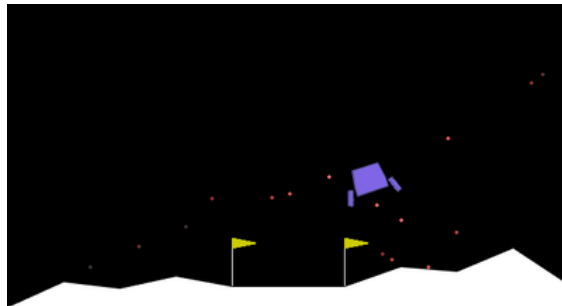


Figure 1: Illustrative LunarLander-v3 frame from one archived audit episode (audited-selection outer seed 101, candidate 9, episode seed 995252232). The episode’s raw return was 190.06. This image is an environment preview, not an additional experiment or the 32-episode mean.

Recipe contract. A candidate is a Python module defining three components: (i) separable CMA-ES settings ($\sigma_0 \in [0.01, 2]$, population size 4–64); (ii) a generation-dependent schedule specifying 1–16 evaluation episodes and a 50–1000 step training horizon; and (iii) a deterministic per-transition shaped-reward function. Shaped reward must be finite and have magnitude at most 10^6 per transition. The candidate cannot change the policy architecture, trainer, total episode budget, audit, or archive.

Generated code passes AST hygiene checks, import/loading checks, and dynamic contract probes. These checks are not a security sandbox; experiments should run on trusted infrastructure. If a proposal fails, the next rejection-sampling prompt includes the validator’s reason. The complete static prompt is hashed into the immutable run configuration.

Fixed inner trainer. The policy is a tanh multilayer perceptron with layers (8, 32, 32, 4), totaling 1,476 parameters. It is trained with separable CMA-ES [2, 13], following the broader use of evolution strategies for policy search [14]. A candidate receives at most 2,000 training episodes and 600 wall-clock seconds. Population members are evaluated on a small, fixed training seed pool under that recipe’s shaped reward and schedule.

Two reward sources. The primary candidate-shaped protocol executes each recipe’s shaped-reward function during training. The fixed-raw control instead substitutes raw environment reward inside the trainer and never calls candidate shaping. Candidates in both protocols can still choose CMA-ES settings, evaluation episodes, and horizon schedules. The control therefore removes candidate-defined reward content and scale together; it does not hold every other source of training noise fixed.

Two selection scores. The *claimed* score is the highest training fitness observed in the candidate’s run. The submitted parameters are also those judged best by that signal. The *audited* score is the mean raw environment return from 32 new seeds drawn from OS entropy, using a fixed 1000-step horizon. Audit code receives the policy parameters but does not import or call the recipe. For a candidate i , we record the honesty gap

$$g_i = c_i - a_i,$$

where c_i is its reported best training score and a_i its audit. Under fixed raw reward, c_i is the raw training return of the submitted policy, but it remains the best noisy estimate observed on reused training seeds. Under candidate shaping, scores need not share units, so g_i is a diagnostic rather than a calibrated estimate of lying or intent.

Outer loop. Each run begins with a hand-written raw-reward seed recipe. The outer loop selects a parent (80% uniformly from the current top five, 20% from the whole completed archive), gives the LLM the parent source and its selection score, validates the proposed child, trains it, audits it, and appends a record. In **audited selection**, parent ranking and feedback use a_i . In **claimed selection**, both use c_i ; audits remain hidden from selection and are recorded only for analysis. After 15 completed candidates, the system returns the archive maximum under the selected signal. Python’s stable ordering resolves exact score ties by earlier archive order.

3 Experimental protocol

Table 1 summarizes the two experimental phases. LunarLander-v3 uses Gymnasium’s discrete four-action task and raw environment return [16]. Protocol v3 uses candidate-shaped training rewards; protocol v4 is the fixed-raw control. Within each phase, we used outer-loop RNG seeds 101, 202, and 303 in both selection modes. The same seed labels align some harness random streams, but neither the within-phase comparison nor the cross-phase ablation is strictly paired: model sampling is stochastic, audit seeds come independently from OS entropy, and the phases were run on different dates with different static prompts.

Item	Candidate-shaped (v3)	Fixed-raw (v4)
Environment	LunarLander-v3, discrete actions	
Selection modes	audited, claimed	
Independent runs	3 per cell (seeds 101, 202, 303)	
Candidates	15 completed candidates per run	
Training budget	2,000 episodes, 600 s per candidate	
Audit	32 fresh seeds, raw reward, 1,000-step horizon	
Training reward	candidate shaped	raw environment reward
Mutation requests	84	84
Prompt fingerprint	be9c5020...a00e8a	f420fda4...107df

Table 1: Experimental design. Both phases used Codex CLI 0.146.0, `gpt-5.6-so1`, and low reasoning. A run, not a candidate or audit episode, is the unit of replication.

Seed	audited-mode pick	claimed-mode pick	claimed-run oracle	actual regret
101	33.0	−1102.3	−111.5	990.8
202	−48.7	−422.7	−85.4	337.3
303	27.6	−761.4	41.6	802.9
Mean	4.0	−762.1	−51.8	710.4

Table 2: Run-level outcomes in raw LunarLander reward. “Oracle” is the best audited candidate already present in the claimed-selection archive; it is post-hoc and was unavailable to that selection rule.

For each run we report: (i) the raw-reward audit of the candidate actually selected by that mode; (ii) the best audit observed anywhere in the same archive (an oracle available only after recording audits); and (iii) selection regret, the oracle audit minus the selected candidate’s audit. Audited selection has zero within-archive regret by construction, so its zero is not an empirical discovery. The informative questions are the audited quality of each mode’s final pick and whether claimed scores preserve enough ordering information to select among their own archive.

We provide no candidate-level p -values: candidates share ancestry, and the 32 audit episodes measure one submitted policy rather than 32 independent evolution runs. With only three outer runs per cell, all between-mode and cross-protocol summaries are descriptive.

4 Results

4.1 Final selected policies

The audit-selected final policies scored 33.0, −48.7, and 27.6 on raw environment reward (mean 4.0). The claim-selected final policies scored −1102.3, −422.7, and −761.4 (mean −762.1). The seed-aligned audited-minus-claimed differences were 1135.3, 374.0, and 789.0 points (Figure 2). These are large realized differences, but three stochastic runs do not establish their population frequency or expected magnitude.

4.2 Claim saturation destroys ranking information

In all three claimed-selection runs, generated recipes reached a claimed score of exactly 10^9 . This value follows from the contract rather than a validator escape: a recipe may emit a shaped reward

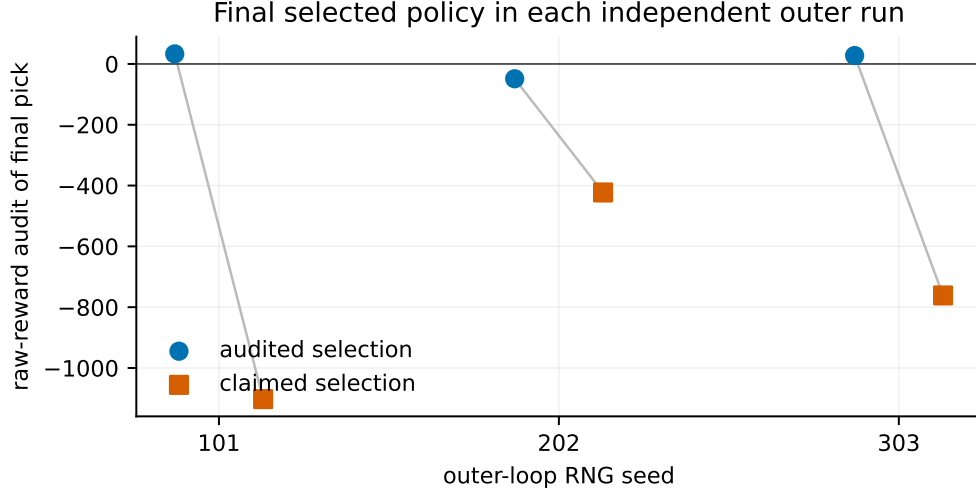


Figure 2: Raw-reward audit of the final candidate returned by each selection rule. Lines align the same outer-loop RNG seed, but model samples and audit seeds are not paired.

of 10^6 at each of 1000 steps. The resulting number is a valid shaped-training return but has no raw-reward interpretation. Candidate-level scores separate sharply by mode in Figure 3a: audited selection still produces optimistic claims, whereas claimed selection drives many candidates to the allowed scale ceiling.

The highest claimed score was tied by 5, 2, and 3 candidates in seeds 101, 202, and 303. The implemented stable tie-break selected the earliest, producing regrets of 990.8, 337.3, and 802.9 (Table 2). To avoid attributing the full result to that arbitrary tie-break, we also compute a favorable bound: retrospectively choose the best audited policy among all candidates tied at the maximum claim. The remaining regrets are 1.2, 337.3, and 273.8 (mean 204.1). Thus, even an audit-informed tie-break among top claims would fail to recover the archive oracle in every run (Figure 3b). Once reports saturate, the claimed metric cannot distinguish policies that differ by hundreds of raw-reward points.

This is a comparability failure, not proof that large shaping always produces a poor policy. One candidate in seed 303 claimed 9.999×10^8 and audited at 41.6, the best raw-reward policy in that archive. Other nearly identical claims audited hundreds of points lower. The issue is precisely that the claimed score cannot tell them apart.

4.3 Fixed-raw reward-source ablation

The fixed-raw phase asks whether candidate-defined reward content and scale are necessary for the selection failure. They are not. Under audit selection, final-policy audits were -6.7 , -69.3 , and 17.9 (mean -19.4). Under claimed selection they were -166.9 , -140.1 , and -104.2 (mean -137.1). In all three claimed-selection runs, another candidate in the same archive had a higher audit. Selection regrets were 172.3, 86.0, and 73.9 (mean 110.7).

Figure 4 supports a layered account. Candidate-defined reward scale is a major amplifier: removing it reduced mean claimed-selection regret from 710.4 to 110.7 and reduced the mean final-policy gap between selection modes from 766.1 to 117.7. It is not the sole source: the fixed-raw claimed score is still the maximum observed training estimate, often based on one episode at the selected best generation, on reused training seeds and candidate-chosen schedules. These features

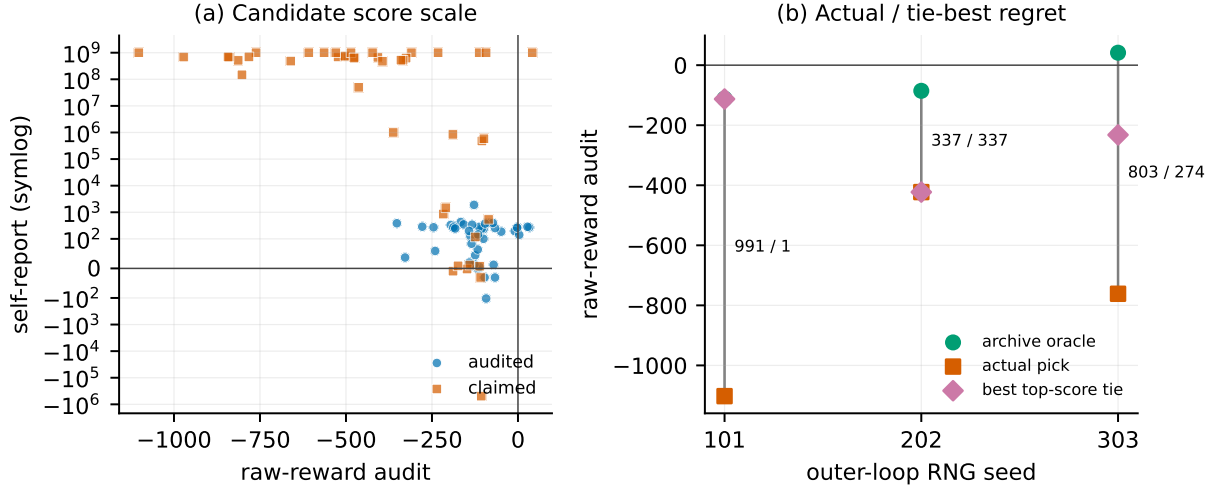


Figure 3: Mechanism evidence. (a) All 90 candidates; the self-report axis uses a symmetric log scale. Points are dependent within outer runs. (b) Within each claimed-selection archive: actual stable-tie pick, best audited candidate among maximum-claim ties, and overall audit oracle. Labels show actual / tie-favorable regret.

Seed	audited-mode pick	claimed-mode pick	claimed-run oracle	regret
101	-6.7	-166.9	5.4	172.3
202	-69.3	-140.1	-54.1	86.0
303	17.9	-104.2	-30.3	73.9
Mean	-19.4	-137.1	-26.3	110.7

Table 3: Fixed-raw control outcomes. “Claimed” now means the best raw training return observed on the training seed pool, not candidate-shaped reward. The oracle remains post-hoc.

are plausible explanations, not separately randomized causal findings. Because the v3 and v4 phases used different prompts and were run at different times, the numerical reduction itself should not be read as a clean paired estimate of reward shaping’s causal effect.

4.4 What the experiment does and does not show

The experiment directly shows that the permitted candidate-controlled metric can saturate and that selecting on it can return a much worse audited policy than one already trained in the same run. The fixed-raw phase further shows that raw training scores can still misrank policies relative to a fresh-seed audit. A harness-owned audit preserves a common unit and separates selection from the training measurement. The study does *not* isolate every remaining cause, prove that audit selection improves search in expectation, or show that selected recipes reproduce their policy quality when retrained.

The repository also contains historical MinAtar Breakout [18] pilot archives using a gymnasium backend [4]. Those pilots have only one outer run per mode and predate protocol-v3 metadata. They demonstrate that the harness supports a second environment and execution backend, but we do not pool them with the primary experiment or claim cross-environment replication.

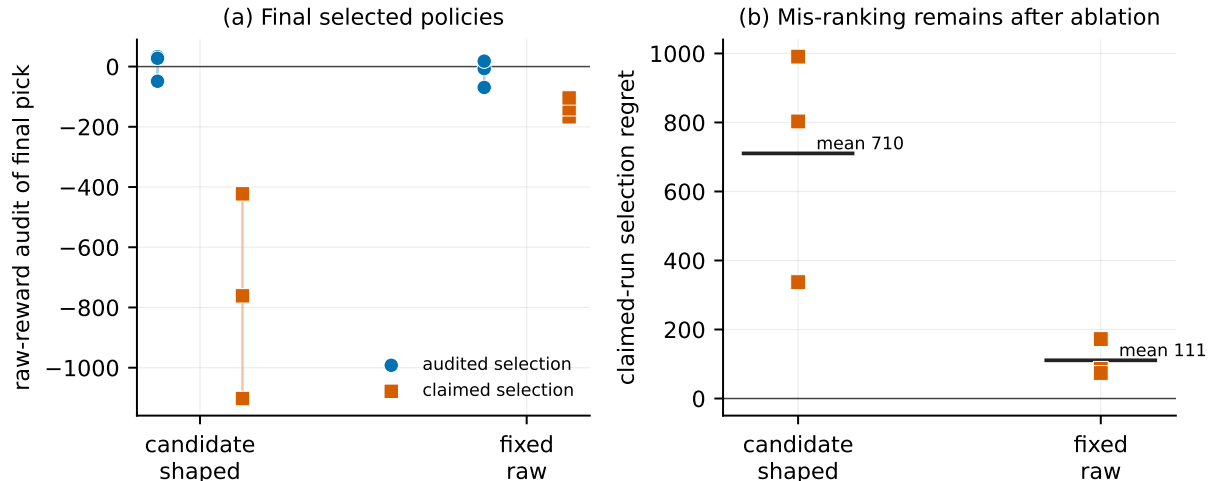


Figure 4: Reward-source ablation. Points are independent outer runs ($n = 3$ per cell); bars show means. Candidate shaping creates score saturation and much larger realized regret. Fixed raw reward removes saturation but not all misranking between noisy training scores and fresh audits.

5 Reproducibility and provenance

Every replicated run stores an immutable configuration, Python and package versions, mutation provider, CLI version, model identifier, reasoning setting, full static-prompt hash, outer-loop RNG checkpoint, complete training history, audit seeds, per-episode raw returns, candidate source, and trained parameters. Rerunning with a changed configuration is rejected rather than silently mixing protocols. The 12 included runs contain exactly 180 completed candidates and no failed records.

Across both phases, the mutation operator made 168 requests totaling 2,988,801 recorded input tokens and 77,263 output tokens; all passed validation on the first attempt. Authentication used a local ChatGPT login, but no secret or credential is stored in the archives. Separate public replication bundles are sanitized for machine-local paths, licensed under Apache-2.0, checked for restricted keywords and secret-like values, and accompanied by per-file hashes plus deterministic archive checksums. The analysis scripts reconstruct Tables 2 and 3 directly from the bundles without executing generated recipes.

6 Limitations

Three runs per cell. The consistency and magnitude observed here motivate further work but do not justify asymptotic, statistical-significance, or generality claims. **One replicated environment.** Breakout remains a historical pilot; the primary evidence is LunarLander only. **Joint rather than surgical ablation.** Fixed raw reward removes both shaping content and scale. It does not distinguish scale normalization from removing shaping altogether. **Not strictly paired.** Reusing outer RNG seed labels does not fix stochastic model outputs or fresh audit seeds. The two phases also used different prompts and dates, so seed-aligned and cross-protocol differences are descriptive rather than randomized paired estimates. **One model configuration.** The recorded identifier is a service model name, not an immutable server-weight snapshot; CLI version, date, prompt hash, and generated sources provide the available provenance. **Audits certify policies,**

not recipes. A fresh-seed audit measures one trained parameter vector. Stable recipe quality would require independent retraining of finalists. **Generated code.** Static checks are hygiene, not adversarial containment. The system should not execute candidates near secrets or untrusted resources.

7 Related work

Program-evolution systems with exact or harness-owned evaluators include FunSearch and AlphaEvolve [9, 12]; AutoML-Zero similarly evaluates evolved algorithms inside a controlled pipeline [11]. Eureka uses LLM-generated reward functions but judges them by policies trained and evaluated under a fixed outer procedure [7], structurally closer to our audited condition. Our claimed condition instead exposes what happens when a candidate-defined training metric becomes the selection metric itself.

The observed failure belongs to the broader family of reward hacking, specification gaming, and Goodhart effects [3, 5, 8, 10, 15]. Automated-science systems have exhibited adjacent failures such as modifying evaluation code [6]. Here candidates need not violate the interface: optimizing an allowed measurement scale is enough. The audit gate is the minimal architectural separation between candidate-controlled optimization and a comparable selection signal.

8 Conclusion

When LLM-generated training recipes control reward shaping, their self-reported fitness values need not share a meaningful scale. Candidate-shaped runs drove reports to the contract ceiling and selected policies whose raw-reward audits were far below alternatives already in the archives. Fixed raw reward removed saturation and sharply reduced the observed damage, but training-score selection still misranked the archive in all three runs. The evidence therefore supports a layered failure: candidate-defined scale can dominate selection, while noisy best-of-training estimates can remain misaligned with fresh-seed policy quality. A separate audit using fresh seeds, raw rewards, and no recipe-code path restores a common selection measurement. Broader claims require more environments, models, outer runs, a randomized reward-scale control, and finalist retrains.

Artifacts. Code, analysis, and release instructions are available at the project repository. Separate protocol-v3 and protocol-v4 data bundles contain the complete replicated evidence and deterministic SHA-256 checksums.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.
- [3] Victoria Kravovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.
- [4] Robert Tjarko Lange. gymmax: A JAX-based reinforcement learning environment library. <https://github.com/RobertTLange/gymmax>, 2022.

- [5] Joel Lehman, Jeff Clune, Dusan Misevic, Christoph Adami, Lee Altenberg, Julie Beaulieu, Peter J. Bentley, Samuel Bernard, Guillaume Beslon, David M. Bryson, et al. The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities. *Artificial Life*, 26(2):274–306, 2020.
- [6] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [7] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s law. *arXiv preprint arXiv:1803.04585*, 2018.
- [9] Alexander Novikov, Ngân Vŭ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [10] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [11] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *International Conference on Machine Learning (ICML)*, 2020.
- [12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [13] Raymond Ros and Nikolaus Hansen. A simple modification in CMA-ES achieving linear time and space complexity. In *Parallel Problem Solving from Nature (PPSN)*, 2008.
- [14] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [15] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [17] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.
- [18] Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. *arXiv preprint arXiv:1903.03176*, 2019.